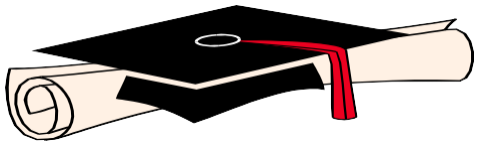
PCS

Programming with C

Semester: 2	**The Exam Code:** 08
Duration: 90 minutes	**Total Marks:** 20



Practical Examination Paper

Do not write on this question paper and return it to the Invigilator after the examination.

C# Programming Exam: Student Course Enrollment System

✅Instructions

- Structure your code clearly and use comments.
- Validate all user inputs.
- Use `List<T>` for storing course/student data.
- Submit all `.cs` files in a zipped folder named:
  `CSharp_Exam_<YourName>.zip`

□ Question 1: Create an Interface (2 Points)

Task:
Create an interface named `IEnrollment` with the following methods:

Return Type	Method Name	Description
`float`	`CalculateTuition()`	Returns total tuition for a student
`void`	`DisplayInfo()`	Displays student course enrollment information

□ Question 2: Create the Student Class (6 Points)

Task:
Create a class `Student` that implements `IEnrollment`.

Fields and Properties:

Field Name	Type	Description
`studentId`	`int`	Unique ID of the student
`fullName`	`string`	Full name of student (5–100 characters)
`creditHours`	`int`	Number of enrolled credit hours (must be ≥ 1)
`ratePerHour`	`float`	Tuition rate per credit hour (> 0)

Requirements:

Constructors:

- A no-argument constructor
- A constructor with all fields and validation

Properties:

- `StudentId` → no validation
- `FullName` → must be 5–100 characters
- `CreditHours` → must be ≥ 1
- `RatePerHour` → must be > 0

Interface Implementation:

- `CalculateTuition()` → `creditHours * ratePerHour` **(2 Points)**
- `DisplayInfo()`
  Example:
  `"Student: John Doe | Hours: 15 | Tuition: $2250"` **(1.5 Points)**

Validation Rules (2.5 Points):

- Full name length: 5–100 characters
- Credit hours ≥ 1
- Rate per hour > 0
- Invalid inputs should throw `ArgumentException` or show messages

□ Question 3: Create ScholarshipStudent Class (Inheritance) (4 Points)

Task:
Create a class `ScholarshipStudent` that extends `Student`.

Additional Field:

- `discountRate`: `float` — percentage discount on tuition (default: 20%)

Override Methods:

- `CalculateTuition()` →
  Formula: `creditHours * ratePerHour * (1 - discountRate / 100)` **(2 Points)**

- `DisplayInfo()`
  Example:
  `"Scholarship Student: Anna | Hours: 12 | Tuition: $1920 (after discount)"` **(2 Points)**

□ Question 4: EnrollmentManager - CRUD Operations (5 Points)

Task:
Create a class `EnrollmentManager` that manages students using a `List<Student>`.

Required Features:

Feature	Description	Points
Add Student / ScholarshipStudent	Input and validate via console	1.5
View All Enrollments	Display all info with `DisplayInfo()`	1.5
Update Tuition Rate	Update rate per hour by studentId	1.0
Delete Student	Remove by studentId, show message if not found	1.0

□ Question 5: Main Class – Interactive Menu (3 Points)

Task:
Create a `Main` class with a loop menu.

Sample Menu:

```
=== STUDENT ENROLLMENT MENU ===
1. Add New Student / Scholarship Student
2. View All Enrollments
3. Update Tuition Rate
4. Delete Student
5. Calculate Tuition by Student ID
6. Exit
```

- Use `try-catch` for user input and validation
- Menu loops until user chooses Exit
- Call methods in `EnrollmentManager` **(3 Points)**

✅Example Output:

```
Student: Mark | Hours: 18 | Tuition: $2700
Scholarship Student: Lily | Hours: 15 | Tuition: $1800 (after
discount)
```

□ Submission Reminder:

Compress and submit all `.cs` files in a folder named:
`CSharp_Exam_<YourName>.zip`